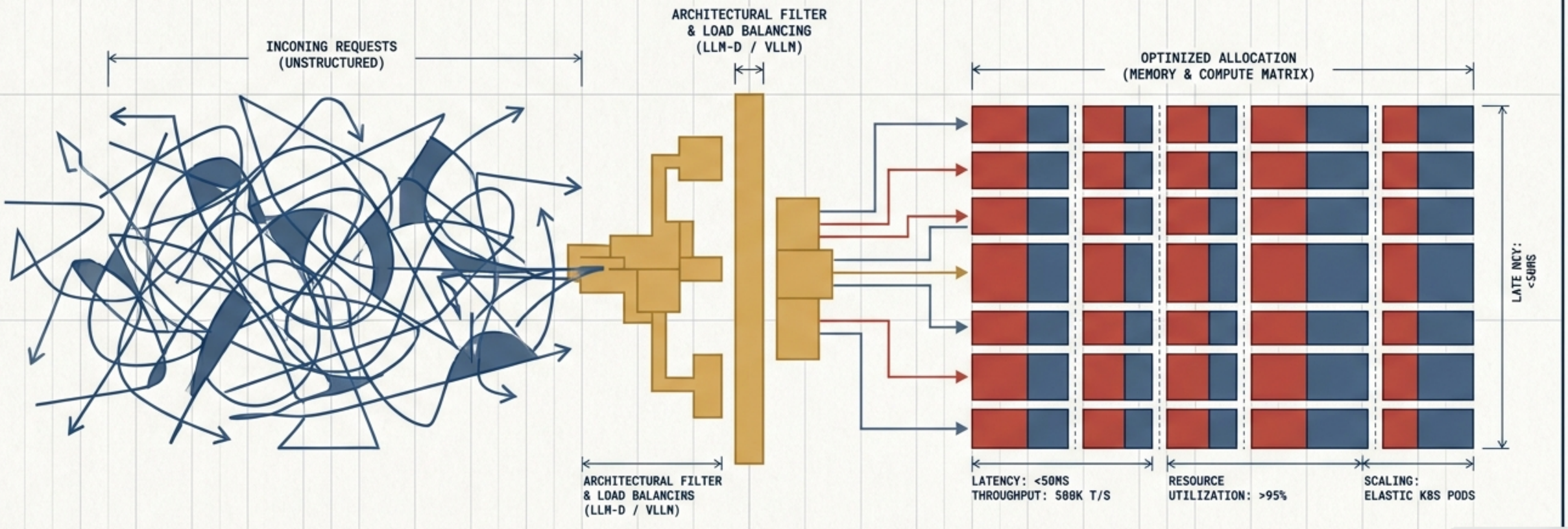
[TARGET ENV: KUBERNETES]
[EVENT: PYCON AFRICA]

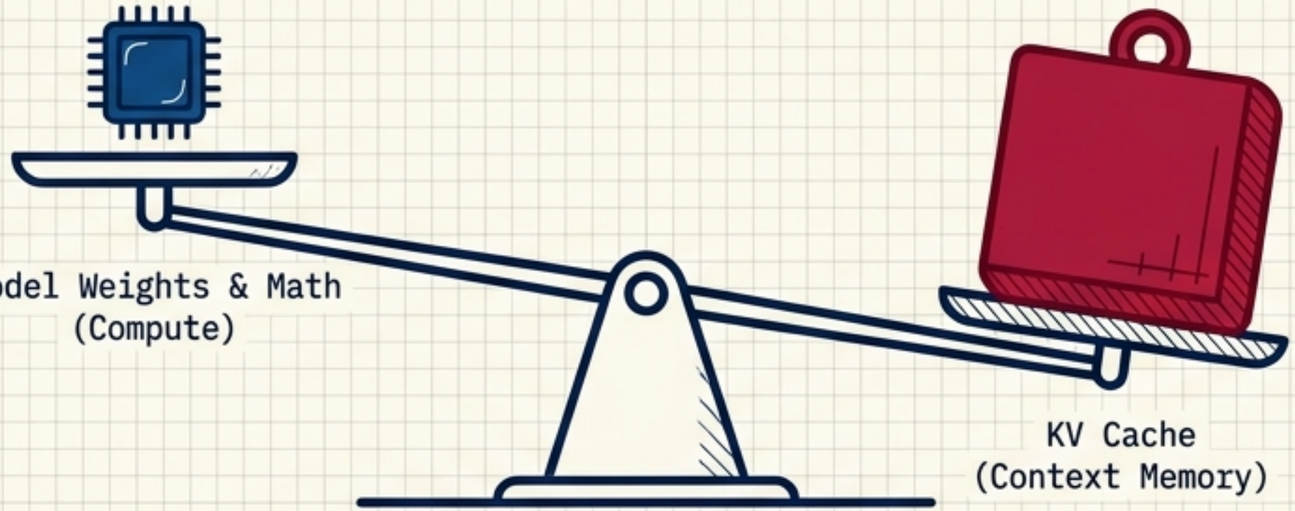
Serving the Unservable

Deploying Custom LLMs at Scale using vLLM and llm-d



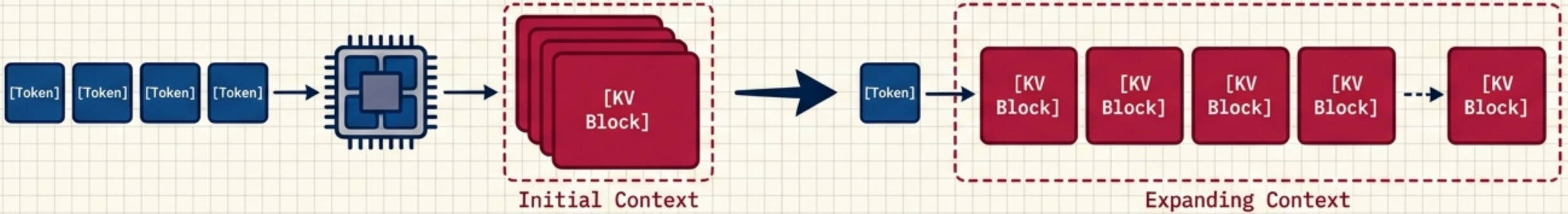
Gemini Notebook

The Core Bottleneck: It's Memory, Not Math



Step 1: Prefill Phase
Reads the prompt.

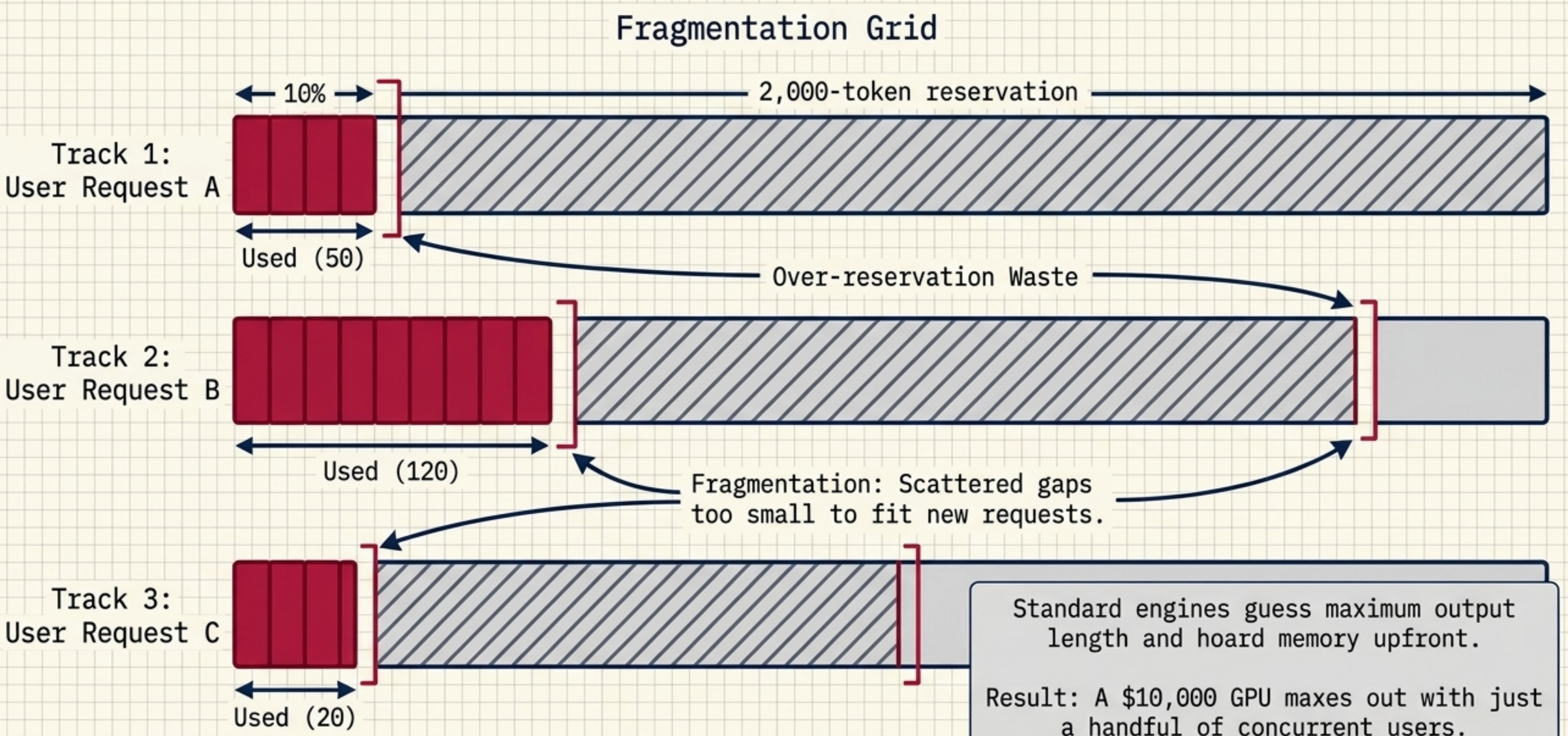
Step 2: Decode Phase
Writes the reply.



Every generated token expands the `**KV Cache**` in GPU memory. Concurrency is entirely dictated by how much memory you have left, not how fast your GPU multiplies matrices.

Gemini Notebook

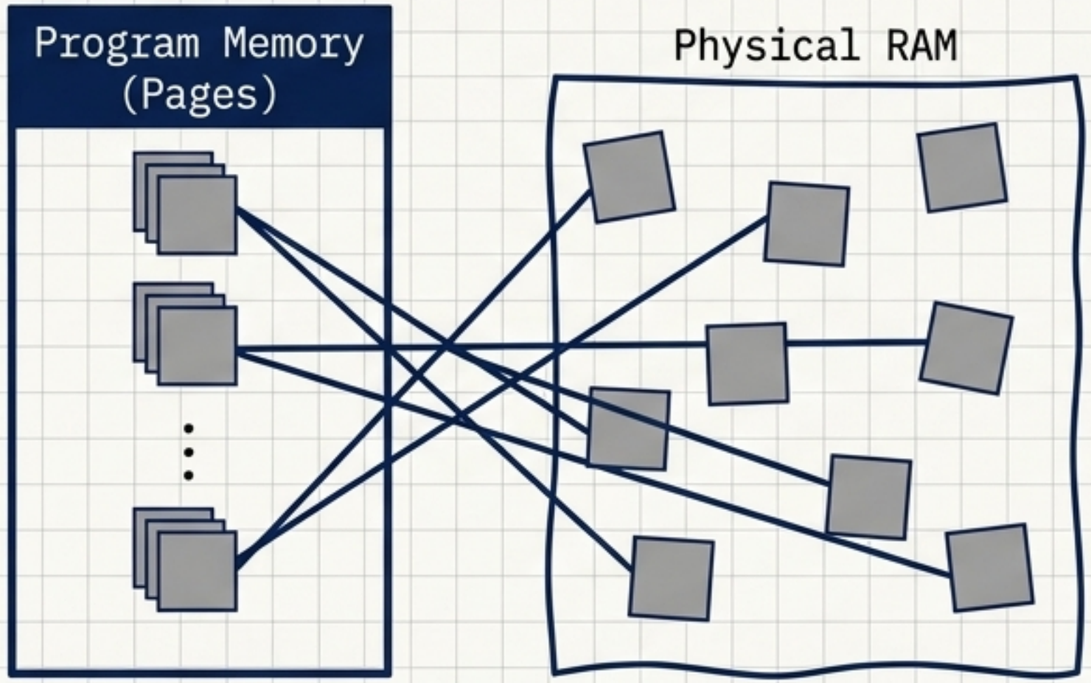
The Naive Serving Trap



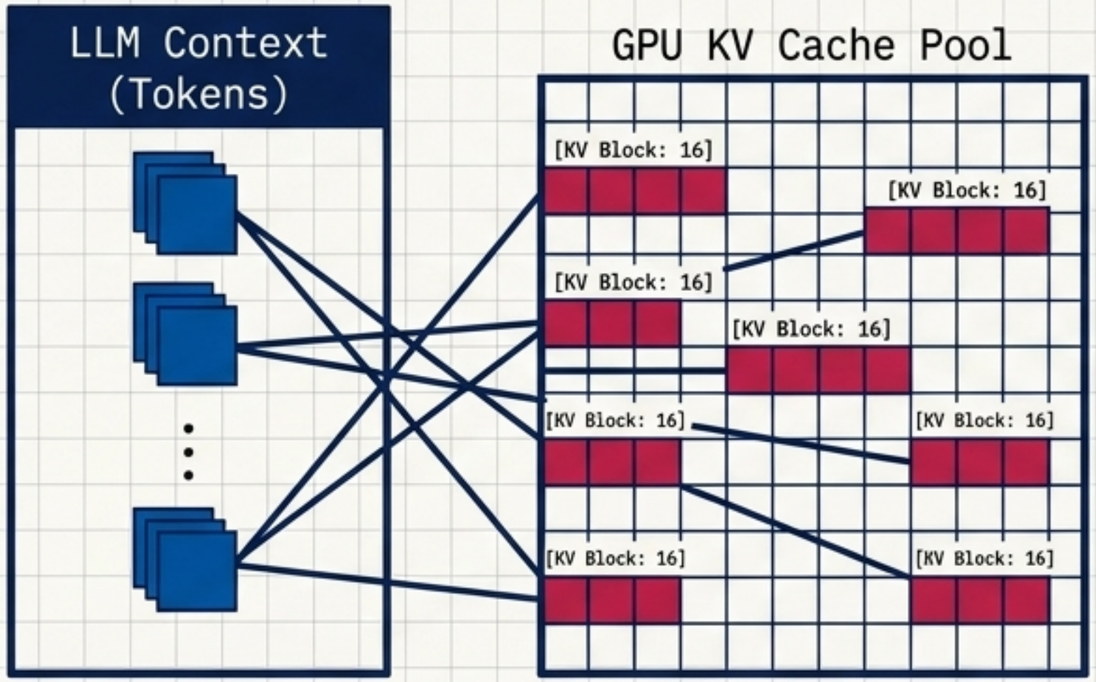
Gemini Notebook

The Paradigm Shift: PagedAttention

OS Virtual Memory



vLLM PagedAttention



1. Divide KV cache into fixed-size blocks (e.g., 16 tokens).

2. Allocate memory on demand via a Block Table. No hoarding.

3. Blocks can be non-contiguous. Zero fragmentation. Waste drops to <4%.

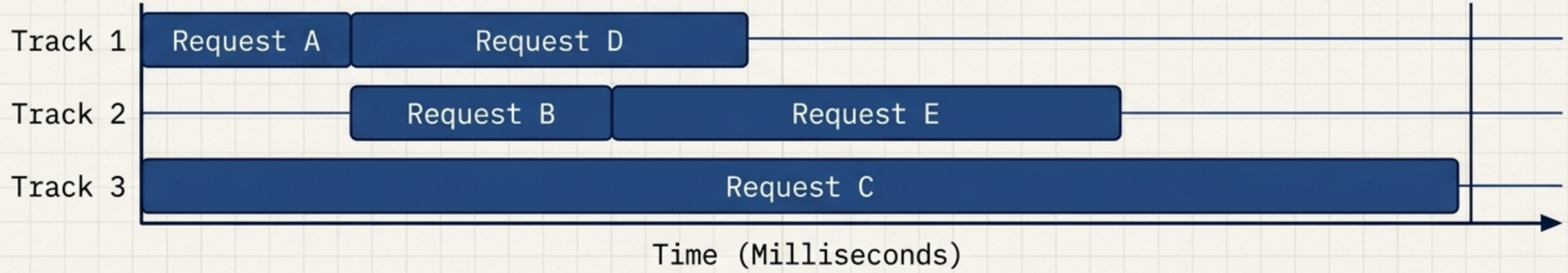
Gemini Notebook

Continuous Batching: Zero Idle Compute

Static Batching (Naive)



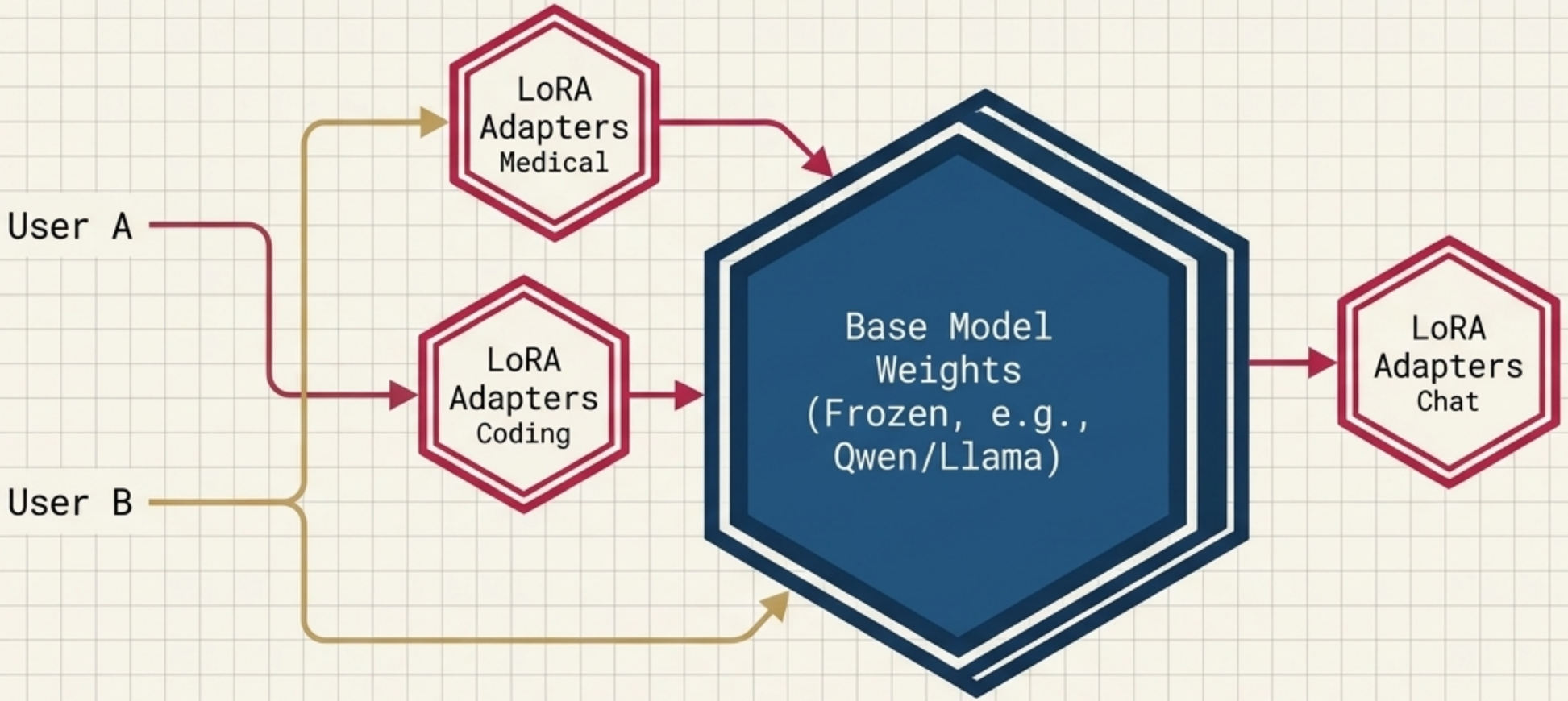
vLLM Continuous Batching



vLLM evaluates token-by-token. The moment a sequence finishes, a new one is swapped in. The GPU never waits for the slowest prompt.

Gemini Notebook

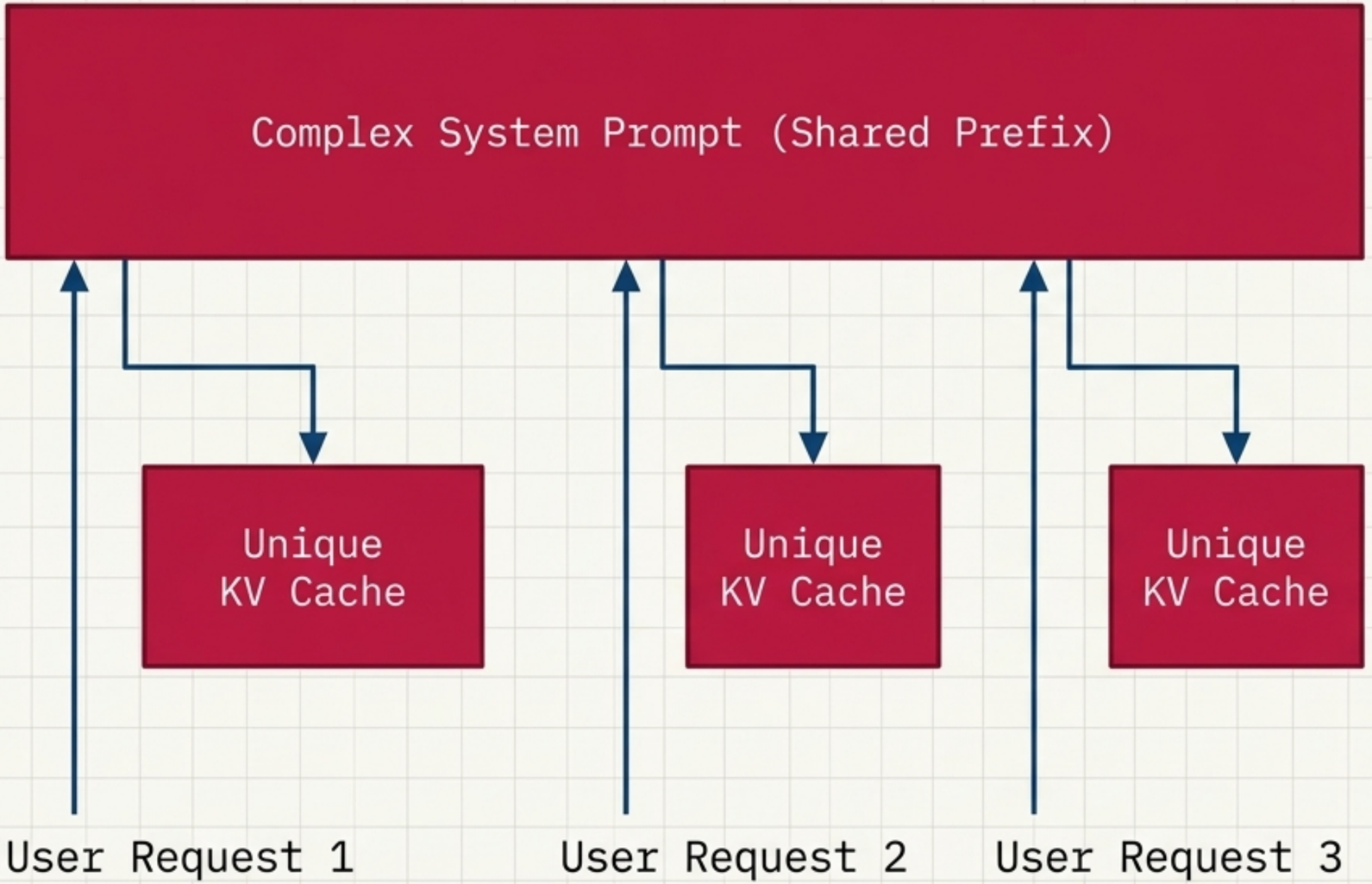
Serving Custom Models: Multi-LoRA Integration



You don’t need a dedicated GPU for every fine-tuned model.
Load the base model once, and dynamically swap lightweight adapters per-request.
One GPU = Dozens of custom models.

Gemini Notebook

Prefix Caching: The Cheat Code for Agentic AI



- **IDENTICAL PREFIXES:** Shared system instructions are stored exactly once in memory.
- **BEAM SEARCH:** Multiple generation paths share the same root context.
- **AGENTIC WORKLOADS:** Multi-turn tool-calling loops reuse the same massive context window automatically.

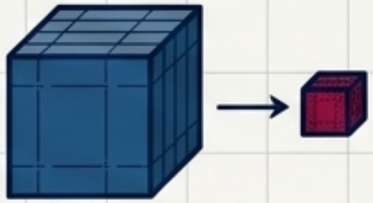
Gemini Notebook

Scaling Up: Quantization & Distributed Inference

Multi-Node Fleet

Y-Axis Hardware Constraints

Quantization



Shrink the weights. Support for INT8, FP8, AWQ, GPQ. GPTQ. Fits larger models on fewer cards.

Pipeline Parallelism



Split model layers sequentially across nodes. High throughput for massive models.

Standard Deployment

FP16 inference on standard hardware.

Tensor Parallelism



Split weight matrices across GPUs. Low latency, requires high-bandwidth interconnects.

Model Size

70B+ Massive

Gemini Notebook

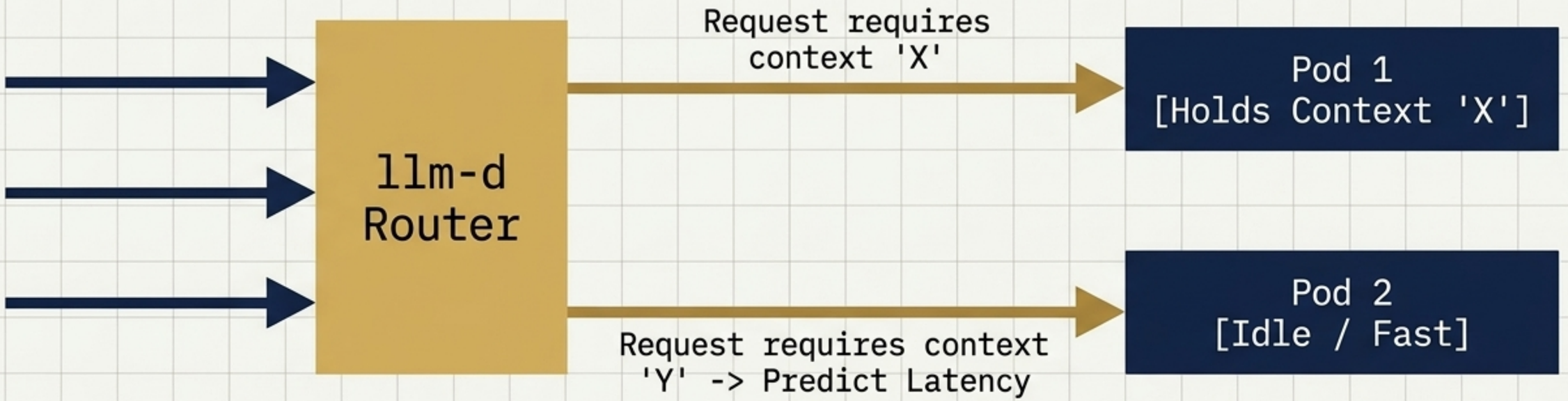
The Missing Link: Fleet-Level Orchestration



vLLM optimizes a single machine perfectly. But traffic spikes 10x, drops to zero, and demands fair scheduling. Round-rob-robin load balancing will crash your optimized engine. Enter Serverless AI.

Gemini Notebook

Introducing llm-d: The Intelligent Gateway

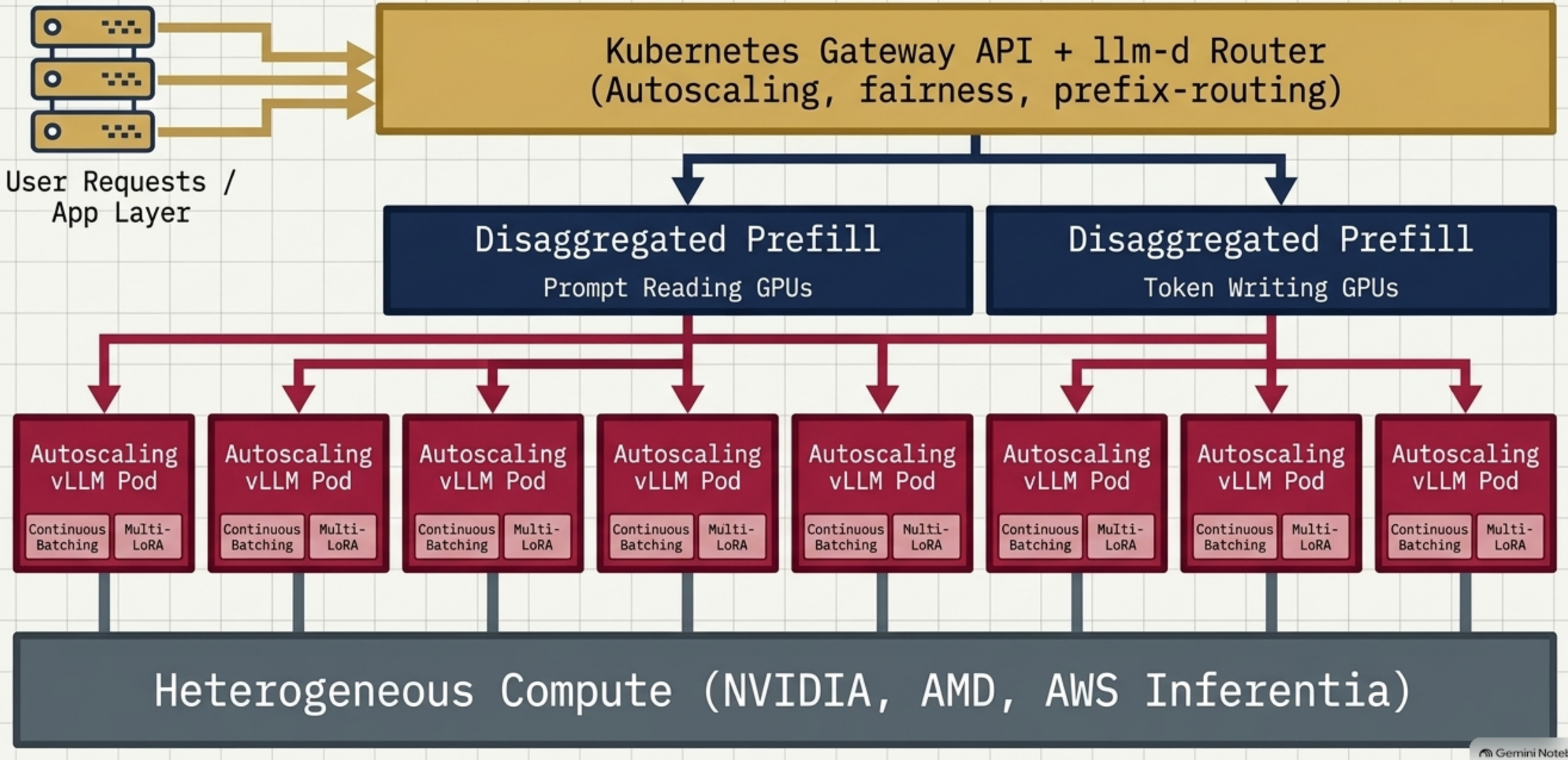


PREDICTED LATENCY ROUTING: Avoids sending requests to pods bogged down by heavy decodes.

PREFIX-CACHE AWARE ROUTING: Sends requests to the exact GPU that already has the prompt cached in memory.

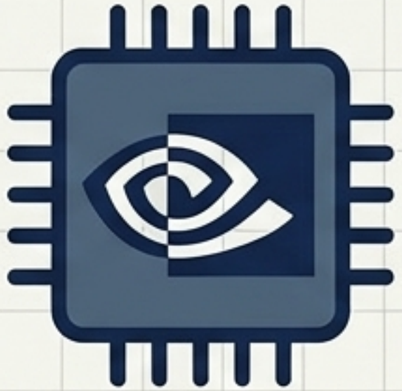
Gemini Notebook

The Serverless Architect's Blueprint



Gemini Notebook

Hardware Agnosticism: Avoid Vendor Lock-in



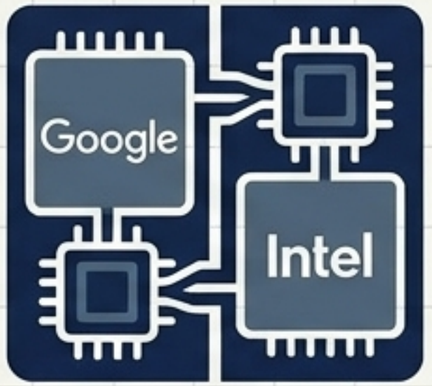
NVIDIA
CUDA / FlashAttention



AMD
ROCm



AWS
Trainium / Inferentia2 (Neuron SDK)



Google / Intel
TPU / Gaudi CPU

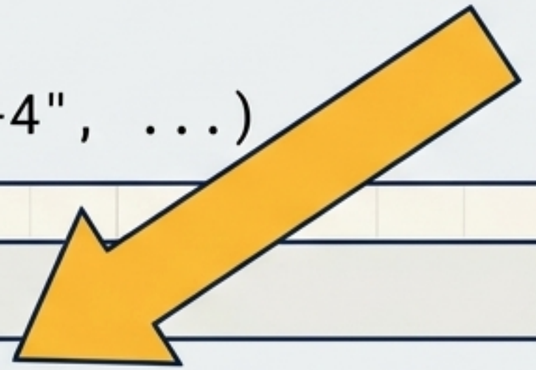
Serverless means dynamically utilizing the best, cheapest compute available. vLLM acts as a unified abstraction layer over diverse hardware accelerators.

Gemini Notebook

The API Abstraction: Drop-In Replacement

Standard Application

```
client = OpenAI(api_key="sk-...")
response = client.chat.completions.create(model="gpt-4", ...)
```



vLLM / llm-d on Kubernetes

```
client = OpenAI(api_key="EMPTY", base_url="http://llm-d-router/v1")
response = client.chat.completions.create(model="qwen3-32b-lora", ...)
```

No application rewrites. vLLM and llm-d expose an OpenAI-compatible HTTP server. Point LangChain, LlamaIndex, or raw API calls to your internal cluster and it instantly works.

Gemini Notebook

Real-World Archetypes

High-Traffic Chatbot

Challenge
High concurrency, varying output lengths.

Stack
vLLM Continuous Batching + llm-d Predicted Latency Routing.

Multi-Tenant B2B SaaS

Challenge
Unique models per client, cost constraints.

Stack
vLLM Multi-LoRA + Quantization (FP8).

Autonomous Agents

Challenge
Massive, repetitive system prompts.

Stack
PagedAttention Prefix Caching + llm-d Cache-Aware Routing.

Gemini Notebook

The Architect’s Checklist

- [x] **Defeat the Memory Bottleneck:** Utilize **PagedAttention** to eliminate KV cache fragmentation.
- [x] **Eliminate Idle Compute:** Deploy **Continuous Batching** for token-level execution.
- [x] **Serve Custom Models Cheaply:** Use **Multi-LoRA** to dynamically swap adapters on a frozen base.
- [x] **Scale Intelligently:** Deploy **llm-d** for cache-aware routing and serverless autoscaling on Kubernetes.

> init deployment

[QR CODE: GitHub Repos]



Gemini Notebook